Foundational Principles of Agentic Security Pipelines: Tool Proxy and Policy Enforcement Architecture

The transition from static, user-facing Large Language Model (LLM) applications to autonomous agentic systems represents a significant shift in enterprise software architecture. While traditional applications typically constrain a model's output to text-based responses, agentic systems empower models to interact with the physical and digital world through tool calling, API execution, and direct database interaction.[1] This increased agency introduces a complex surface area for security vulnerabilities, necessitating a specialized defense-in-depth framework known as the Agentic Security Pipeline.[1] At its core, this pipeline consists of two symbiotic components: a Tool Proxy, which acts as a secure intermediary for all external requests, and a Policy Enforcement Layer, which serves as a deterministic guardrail system for evaluating the legitimacy and safety of inputs and outputs.[1] Understanding these components requires a multifaceted exploration of API gateway patterns, machine-to-machine authentication, LLM-specific threat taxonomies, and the principles of Zero Trust Architecture.[1]

Architectural Patterns for Secure Intermediation

The Tool Proxy is fundamentally a specialized iteration of the API Gateway pattern, designed to intercept, authenticate, and validate every request made by an AI agent to external tools or data stores.[1] In microservices architectures, a gateway provides a single entry point for cross-cutting concerns such as rate limiting, routing, and protocol translation, effectively insulating backend services from the complexities of the calling client.[5] For agentic systems, the proxy must go beyond simple traffic management to perform deep schema validation, ensuring that the model-generated arguments conform to the strict requirements of the target API.[1]

Distinctions in Proxy and Gateway Architectures

Traditional network architectures distinguish between forward proxies, reverse proxies, and API gateways. A reverse proxy, such as NGINX, primarily serves to protect backend servers by hiding their IP addresses, terminating SSL/TLS connections, and distributing incoming load.[4] However, the reverse proxy is generally application-agnostic; it does not "understand" the semantic content of the traffic it directs.[4] In contrast, the Tool Proxy operates as an intelligent "Front Door" that is application-aware, capable of parsing REST or gRPC requests and enforcing complex security and compliance policies based on the agent's intent.[4]

Architectural Component	Primary Function	Relevance to Agentic Security
Reverse Proxy	Network-level traffic forwarding and server protection.[4]	Basic interception of agent-to-tool requests.[1]
API Gateway	Application-aware governance, auth, and routing.[4]	Specialized management of LLM tool calls.[1]
Service Mesh	Internal "East-West" service-to-service communication.[6]	Reliability and observability within the tool ecosystem.[6]
Ingress Controller	Kubernetes-specific traffic routing.[7]	Exposing internal tool services to the agent.[7]

The adoption of the Envoy Proxy as a foundation for such a system provides a high-performance, filter-based architecture.[8] Envoy's L7 HTTP filter stack allows developers to chain together various processing steps, such as rate limiting, JWT validation, and RBAC enforcement, before a request ever reaches the upstream cluster.[8] This modularity is essential for an agentic pipeline where new tools and policies must be integrated without re-engineering the entire gateway.[9]

Resilience and Fault Tolerance through Circuit Breaking

In an environment where an LLM agent may call dozens of external services, the failure of a single downstream tool can lead to cascading system instability. The Circuit Breaker pattern is a vital mechanism for handling these failures gracefully.[1] Much like its electrical namesake, a circuit breaker in a proxy environment monitors for failure rates and "opens" the circuit when a threshold is exceeded, immediately rejecting subsequent requests to the failing service to allow it time to recover.[12] Envoy implements this through distributed counters that track active connections and pending requests.[12]

When a circuit breaker is triggered, the system can apply back-pressure to the LLM agent, signaling that a tool is temporarily unavailable.[12] This prevents the agent from entering infinite retry loops that could exhaust model tokens or infrastructure resources. The state machine of a circuit breaker typically moves from "Closed" (normal operation) to "Open" (rejecting requests) and eventually to "Half-Open" (allowing a limited number of probe requests to test service health).[1]

Machine-to-Machine Authentication and Access

Control

Autonomous agents operate in a non-interactive capacity, meaning traditional user-based authentication flows—such as those involving redirects and consent screens—are inapplicable.[13] Instead, the Agentic Security Pipeline relies on the OAuth 2.0 Client Credentials flow for service-to-service authorization.[1] In this model, the agent application functions as a "confidential client" that proves its identity to an authorization server using a Client ID and a Client Secret.[13]

The OAuth 2.0 Client Credentials Lifecycle

The Client Credentials flow is optimized for background processes and daemons.[16] Because no human user is involved, the permissions (scopes) are granted directly to the application itself by an administrator.[16] When the agent service requires access to a tool, it sends a POST request to the token endpoint of the authorization server, which, upon successful validation, responds with an access token—usually a JSON Web Token (JWT).[13]

JWT Claim	Definition	Importance in Agentic Pipelines
iss	Issuer of the token.[13]	Ensures the token came from a trusted authority.[16]
sub	Subject of the token.[13]	Identifies the specific agent or service account.[15]
aud	Intended audience.[13]	Prevents the token from being used at the wrong gateway.[13]
exp	Expiration timestamp.[13]	Enforces short-lived access to minimize risk.[15]
scope	Granted permissions.[13]	Defines which specific tools the agent can call.[15]

The use of short-lived bearer tokens is a critical security control. Unlike static API keys, JWTs carry cryptographically signed metadata and expire automatically, requiring the client to re-authenticate periodically.[15] This limits the window of opportunity for an attacker who might compromise a token. Furthermore, the Tool Proxy must always validate the JWT's signature and claims before granting access, never implicitly trusting tokens even within internal networks.[13]

Role-Based Access Control (RBAC) and Hierarchies

While OAuth 2.0 handles the "Who" and "How" of authentication, Role-Based Access Control (RBAC) manages the "What" of authorization.[17] The NIST RBAC standard provides a structured model for defining roles, permissions, and sessions.[17] In a security pipeline, tools are treated as resources, and specific operations (e.g., read_file, delete_record) are assigned to roles such as Support_Agent or Data_Scientist.[17]

Hierarchical RBAC adds a layer of seniority to these roles, allowing a "Manager" role to inherit all permissions of an "Employee" role.[19] This is particularly useful in agentic systems where specialized sub-agents might have restricted roles, while a primary orchestrator agent possesses broader, inherited permissions.[19] Static and Dynamic Separation of Duties (SoD) further harden the system by ensuring that a single agent cannot hold mutually exclusive roles—for example, preventing an agent from both initiating a financial transaction and approving it in the same session.[20]

The Threat Landscape: OWASP Top 10 for LLMs (2025)

Securing agentic systems requires a deep understanding of the unique vulnerabilities introduced by Large Language Models. The 2025 OWASP Top 10 for LLM Applications serves as the definitive guide for identifying these risks.[22] While traditional web vulnerabilities like Broken Access Control (A01:2025) and Security Misconfiguration (A02:2025) remain relevant to the pipeline infrastructure, the LLM-specific list addresses the probabilistic nature of model interactions.[2]

Critical Risks in Agentic Architectures

The most pressing risk for tool-calling agents is Excessive Agency (LLM06:2025), where an agent is granted too much autonomy or overly broad permissions.[2] This often occurs when developers provide an LLM with access to "god-mode" tools or fail to enforce granular RBAC at the gateway.[22] Similarly, Improper Output Handling (LLM05:2025) can lead to XSS or command injection if a model's output is blindly trusted by downstream systems.[22]

OWASP Risk	Description	Mitigation Strategy
Prompt Injection	Malicious input manipulating model behavior.[22]	Input/Output filtering and intent validation.[24]
Sensitive Info Disclosure	Accidental leakage of PII or secrets.[22]	Egress sanitization and PII detection.[26]

Supply Chain	Compromised models or third-party tools.[22]	Component inventory and provenance checks.[28]
Unbounded Consumption	Resource exhaustion and cost spikes.[22]	Per-tool rate limiting and quotas.[22]
Excessive Agency	Unauthorized autonomous actions.[22]	Principle of Least Privilege and RBAC.[1]

The 2025 update also highlights System Prompt Leakage (LLM07:2025) and Vector/Embedding Weaknesses (LLM08:2025).[2] As RAG systems become the standard for grounding AI outputs, the security of the vector database and the retrieval mechanism becomes paramount. If an attacker can manipulate the retrieved context, they can indirectly influence the model's behavior without ever touching the user prompt.[2]

Taxonomy of Prompt Injection: Direct and Indirect Attacks

Prompt injection remains the paramount security challenge for LLMs, evolving from simple user-submitted overrides to sophisticated indirect attacks.[22] A Direct Prompt Injection occurs when a user explicitly instructs the model to "ignore all previous instructions" and perform a prohibited action.[24] However, in agentic workflows, Indirect Prompt Injection (IPI) is a far more insidious threat.[32]

The Mechanics of Indirect Injection

In an indirect attack, the malicious instructions are placed in external data sources—such as a website the agent summarizes or an email it processes.[30] When the agent retrieves this data, the LLM often fails to distinguish between the developer's original system instructions and the untrusted data it has just ingested.[34] This blurring of the line between "instruction" and "data" allows the external source to effectively hijack the agent's session.[32]

Research by Greshake et al. (2023) demonstrates that these attacks can lead to full system compromise, including the exfiltration of user data to attacker-controlled servers.[32] In one demonstration, a website contained an injection hidden in zero-font-size text that instructed Bing Chat to behave as a social engineer and seek out the user's real name.[33] Because the model is "eager" to follow instructions, it treats the hidden text as an authoritative command.[34]

Strategic Mitigations: Spotlighting and Filtering

Defending against prompt injection requires more than simple keyword blocking, as attackers can use obfuscation, Base64 encoding, or "cognitive hacking" techniques (e.g., urgency, authority claims) to bypass filters.[25] One promising defense is "spotlighting," which uses transformations like delimiting or encoding to help the model distinguish the provenance of

different input sources.[34] By marking untrusted content with unique tokens, developers can instruct the model to treat that specific text as data only, never as instructions.[34]

Additional layers of defense include:

1. **Instruction Hierarchy:** Clearly defining which instructions (system vs. user vs. retrieved) take precedence.[25]
2. **Output Filtering:** Scanning the model's generated tool calls for unexpected behavior or unauthorized tool usage before execution.[1]
3. **Human-in-the-Loop:** Requiring manual approval for high-risk actions, such as data exports or financial transactions.[31]

Policy Enforcement via OPA and Rego

To achieve a deterministic security layer, the Policy Enforcement Layer should be decoupled from the core application logic.[38] The Open Policy Agent (OPA) is a leading solution for this, using a high-level declarative language called Rego to express policies as code.[38] OPA allows developers to offload policy decisions by providing structured JSON data as input, which OPA then evaluates against its ruleset.[38]

Rego: A Declarative Approach to Security

Rego is inspired by Datalog and optimized for fast evaluation of hierarchical data structures.[39] Unlike imperative languages that require explicit loops, Rego uses implicit iteration, automatically finding variable bindings that satisfy a set of conditions.[38] This makes it particularly powerful for checking tool calls against complex allow-lists or verifying that an agent's role permits a specific operation.[38]

Code snippet

```
# Example Rego rule for tool-level access control
allow if {
  input.agent_role == "financial_bot"
  input.tool_name == "retrieve_balance"
  input.args.account_id == input.session_id
}
```

This rule demonstrates how OPA can enforce per-session and per-role constraints.[40] If any condition fails—for instance, if a financial_bot attempts to access an account_id that does not match the current session_id—the result is "undefined," which the Tool Proxy interprets as a

denial.[38] OPA's ability to handle external data via bundles or real-time JWT decoding ensures that policies can be context-aware, incorporating the latest threat intelligence or user-permission updates.[38]

Integration with NeMo Guardrails and Guardrails AI

While OPA provides a general-purpose engine, specialized frameworks like NVIDIA NeMo Guardrails and Guardrails AI offer tools tailored specifically for LLM applications.[41] NeMo Guardrails uses the Colang language to define conversation flows and topical "rails".[43] This allows architects to define a state machine for agent interactions, ensuring the model stays within its intended domain.[44]

Feature	OPA / Rego	NeMo Guardrails	Guardrails AI
Logic Type	Declarative Rules.[38]	Dialog Flows (Colang).[43]	Pydantic-style Validators.[42]
Best Use Case	RBAC, API permissions.[40]	Topicality, Jailbreak prevention.[41]	Format validation, PII scrubbing.[44]
Extensibility	High (Domain Agnostic).[38]	Converational/LLM specific.[41]	Risk-focused (Guardrails Hub).[42]
Enforcement	Decoupled Decision.[38]	Parallel Execution Engine.[45]	Intercept/Retry Logic.[46]

When combined, these tools provide a comprehensive defense. NeMo can handle conversational topicality, OPA can enforce strict API access rules, and Guardrails AI can perform final checks on the structured output to ensure format compliance and the absence of toxic content.[44]

Privacy and Data Minimization: Microsoft Presidio

Data privacy is a core requirement of the Policy Enforcement Layer.[1] Personally Identifiable Information (PII) must be detected and de-identified before being sent to an LLM or an external tool backend.[1] Microsoft Presidio provides an industry-standard framework for automated PII detection and anonymization, leveraging both rule-based pattern matching and Natural Language Processing (NLP).[26]

The Analyzer and Anonymizer Engines

The Presidio Analyzer identifies candidate PII spans—such as phone numbers, email addresses, and credit card numbers—and labels them with entity types and confidence scores.[27] This

process is highly extensible; developers can add custom PatternRecognizer classes using regular expressions for domain-specific identifiers like customer IDs or internal ticket numbers.[26]

The Anonymizer then takes these identified spans and applies a transformation operator.[27] The choice of operator is governed by the trade-off between data utility and privacy.[48]

Operator	Action	Trade-off
Replace	Swaps PII for a tag (e.g., <PHONE_NUMBER>).	High utility for the LLM; low privacy.[48]
Redact	Completely removes the text.	Highest privacy; can destroy semantic context.[48]
Mask	Replaces characters with a symbol (e.g., *).	Good for partial verification.[48]
Encrypt	Cryptographically transforms the string.	Reversible (with the right key); breaks LLM flow.[48]

Implementing Presidio within the pipeline ensures that sensitive data never leaves the organization's control in the clear.[49] To prevent "data leakage" through audit logs, the pipeline should implement the GDPR principle of data minimization, logging only content hashes or already-anonymized strings.[1]

Security Architecture and Trust Boundaries

The design of the Tool Proxy and Policy Engine must adhere to the principles of Zero Trust Architecture (ZTA), as outlined in NIST SP 800-207.[3] Traditional perimeter security, which trusts any entity inside the corporate network, is insufficient for cloud-native agentic systems.[3] Instead, ZTA emphasizes "never trust, always verify," treating every access request as potentially malicious regardless of its origin.[51]

PDP and PEP: The Logical Core of Zero Trust

NIST 800-207 defines three core logical components for implementing Zero Trust [51]:

1. **Policy Decision Point (PDP):** Comprised of the Policy Engine and Policy Administrator, this component gathers telemetry (identity, device posture, behavior) to approve or deny requests.[53]
2. **Policy Enforcement Point (PEP):** The "gatekeeper" that enables, monitors, and

terminates connections.[53] In our architecture, the Tool Proxy serves as the PEP.

3. **Policy Information Point (PIP):** Sources of truth like ICAM systems or threat intelligence feeds that inform the PDP's decisions.[53]

Component	Responsibility	Implementation in Pipeline
Policy Engine (PDP)	Makes the ultimate access decision.[54]	OPA / NeMo Guardrails.[1]
Policy Admin (PDP)	Translates decisions into actionable commands.[54]	FastAPI Controller.[1]
Policy Enforcement (PEP)	Applies the decision at the resource boundary.[54]	Tool Proxy (Envoy).[1]

This architecture ensures that the Policy Engine determines legitimacy while the Tool Proxy enforces structure and permission, creating a layered defense-in-depth model.[1]

Threat Modeling with STRIDE

To identify vulnerabilities at each trust boundary, security architects use the STRIDE framework.[56] STRIDE—Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege—provides a structured methodology for analyzing data flows.[58]

For the Tool Proxy (Boundary B: privileged execution), a primary threat is Elevation of Privilege, where an unprivileged agent attempts to execute a tool with administrative rights.[1] For the Policy Engine (Boundary A: untrusted content), the focus is on Information Disclosure and Tampering, ensuring that malicious prompts cannot leak system secrets or alter the engine's ruleset.[1]

Infrastructure and Implementation Best Practices

The physical deployment of the Agentic Security Pipeline requires a focus on isolation and observability.[1] Containerization with Docker provides essential namespace isolation, ensuring that processes within one container cannot see or affect another.[61] Security best practices for Docker include running containers in rootless mode, pinning base image versions, and dropping all unnecessary kernel capabilities (--cap-drop all).[62]

FastAPI and Dependency Injection

For the API development of the Tool Proxy and Policy Engine, FastAPI is the recommended

framework due to its native support for asynchronous operations and its powerful dependency injection system.[1] In FastAPI, the Depends feature is used to manage cross-cutting concerns like authentication and database sessions.[65]

Using Depends for authentication is preferred over global middleware because it provides granular, per-route control.[67] For instance, a login route can be kept public, while all tool-calling routes require a get_current_user dependency that validates the OAuth 2.0 token.[67] This prevents code repetition and ensures that security invariants are strictly enforced before the route handler ever executes.[68]

Structured Logging with NDJSON

Observability is maintained through structured logging in the Newline Delimited JSON (NDJSON) format.[1] NDJSON (also known as JSONL) stores one JSON object per line, which is ideal for streaming large datasets and processing records incrementally.[70] This format is particularly beneficial for audit logs: if the system crashes, only the last line is potentially lost, unlike a standard JSON array which would become corrupted.[72] Furthermore, NDJSON allows for efficient memory usage, as log parsers can read files line-by-line rather than loading a massive 5GB array into the heap.[72]

Audit logs should capture every step of the agent's interaction:

- **Input:** The raw prompt received from the user.
- **Policy Decision:** Whether the prompt was sanitized, blocked, or allowed.
- **Tool Call:** The structured request generated by the LLM.
- **Gateway Enforcement:** Validation of the schema and agent permissions.
- **Result:** The data returned by the tool, prior to and after post-processing sanitization.

This level of detail enables the creation of a /history endpoint that provides a complete, auditable trace of the agent's behavior, supporting both debugging and compliance requirements.[1]

Conclusion: Strategic Integration of Pipeline Components

Building an Agentic Security Pipeline is a complex undertaking that requires integrating disparate technologies into a cohesive, high-performance architecture.[1] The success of the pipeline depends on the "Inextricable Connection" between the Tool Proxy and the Policy Enforcement Layer; the proxy cannot enforce permissions without the engine's legitimacy decision, and the engine's decision is meaningless without the proxy's enforcement at the resource boundary.[1]

By mastering these foundational concepts—from the nuances of Rego policy authoring to the intricacies of OAuth 2.0 M2M flows—engineers can build systems that provide autonomous

agents with the "agency" they need to be useful, while maintaining the "security" required to be safe in an enterprise environment.[1] The future of AI integration lies in these specialized, programmable gatekeepers that treat every interaction as an untrusted, yet verifiable, transaction.[3]

Works cited

1. # Project Plan: Tool Proxy & Policy Enforcement Layer
2. OWASP Top 10 LLM Risks 2025: Key AI Security Updates - Qualys Blog, accessed March 20, 2026, https://blog.qualys.com/vulnerabilities-threat-research/2024/11/25/ai-under-the-microscope-whats-changed-in-the-owasp-top-10-for-llms-2025
3. What Is NIST SP 800-207? zero trust Architecture Framework - Palo Alto Networks, accessed March 20, 2026, https://www.paloaltonetworks.com/cyberpedia/what-is-nist-sp-800-207
4. Is an API Gateway a Reverse Proxy? - API7.ai, accessed March 20, 2026, https://api7.ai/blog/api-gateway-vs-reverse-proxy
5. What Is an API Gateway? - Palo Alto Networks, accessed March 20, 2026, https://www.paloaltonetworks.com/cyberpedia/what-is-api-gateway
6. Microservices Patterns : API Gateway Pattern | by Abhinav Thakur | Medium, accessed March 20, 2026, https://medium.com/@abhi.strike/microservices-patterns-api-gateway-pattern-5c4567370a54
7. Reverse Proxy vs. Ingress Controller vs. API Gateway - Traefik Labs, accessed March 20, 2026, https://traefik.io/blog/reverse-proxy-vs-ingress-controller-vs-api-gateway
8. HTTP filters — envoy 1.38.0-dev-069450 documentation, accessed March 20, 2026, https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/http/http_filters
9. What Is Envoy Proxy? Concepts, Architecture & Use Cases - Solo.io, accessed March 20, 2026, https://www.solo.io/topics/omni/envoy-proxy
10. Listener filters — envoy 1.38.0-dev-9bbf94 documentation, accessed March 20, 2026, https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/listeners/listener_filters
11. Matching Filter Chains in Listeners - Envoy proxy, accessed March 20, 2026, https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/advanced/matching/matching_listener
12. Circuit breaking — envoy 1.38.0-dev-65cd7b documentation, accessed March 20, 2026, https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/circuit_breaking
13. Understanding the OAuth 2.0 Client Credentials flow — WorkOS Guides, accessed March 20, 2026, https://workos.com/guide/client-credentials
14. Client Credentials Flow - Auth0 Docs, accessed March 20, 2026,

https://auth0.com/docs/get-started/authentication-and-authorization-flow/client-credentials-flow
15. Client credentials flow in OAuth 2.0: How it works - Scalekit, accessed March 20, 2026, https://www.scalekit.com/blog/client-credentials-flow-oauth
16. OAuth 2.0 client credentials flow on the Microsoft identity platform, accessed March 20, 2026, https://learn.microsoft.com/en-us/entra/identity-platform/v2-oauth2-client-creds-grant-flow
17. Role-based access control - Wikipedia, accessed March 20, 2026, https://en.wikipedia.org/wiki/Role-based_access_control
18. What Is Role-Based Access Control (RBAC)? - IBM, accessed March 20, 2026, https://www.ibm.com/think/topics/rbac
19. The NIST Model for Role-Based Access Control: - Towards A Unified Standard - Ravi Sandhu David Ferraiolo and Richard Kuhnt - CSRC, accessed March 20, 2026, https://csrc.nist.rip/rbac/sandhu-ferraiolo-kuhn-00.pdf
20. Role-Based Access Control (RBAC): A Comprehensive Guide - Pathlock, accessed March 20, 2026, https://pathlock.com/blog/role-based-access-control-rbac/
21. What is role-based access control (RBAC)? - SailPoint, accessed March 20, 2026, https://www.sailpoint.com/identity-library/what-is-role-based-access-control
22. OWASP Top 10 for LLMs 2025: Key Risks and Mitigation Strategies - Invicti, accessed March 20, 2026, https://www.invicti.com/blog/web-security/owasp-top-10-risks-llm-security-2025
23. Introduction - OWASP Top 10:2025, accessed March 20, 2026, https://owasp.org/Top10/2025/0x00_2025-Introduction/
24. OWASP Top 10 LLM, Updated 2025: Examples and Mitigation Strategies - Oligo Security, accessed March 20, 2026, https://www.oligo.security/academy/owasp-top-10-llm-updated-2025-examples-and-mitigation-strategies
25. What is Prompt Injection? - CrowdStrike, accessed March 20, 2026, https://www.crowdstrike.com/en-us/cybersecurity-101/cyberattacks/prompt-injection/
26. Identifying Personal Identifiable Information (PII) in Unstructured Data with Microsoft Presidio, accessed March 20, 2026, https://www.statcan.gc.ca/en/data-science/network/identifying-personal-identifiable-information
27. Microsoft Presidio: an engineer's introduction to PII detection and de-identification - Medium, accessed March 20, 2026, https://medium.com/neural-engineer/microsoft-presidio-an-engineers-introduction-to-pii-detection-and-de-identification-6a7c3fed6e50
28. The 2025 OWASP Top 10: Addressing Software Supply Chain and LLM Risks With Cycode, accessed March 20, 2026, https://cycode.com/blog/the-2025-owasp-top-10-addressing-software-supply-chain-and-llm-risks-with-cycode/
29. From Token Bucket to Sliding Window: Pick the Perfect Rate Limiting Algorithm -

API7.ai, accessed March 20, 2026, https://api7.ai/blog/rate-limiting-guide-algorithms-best-practices
30. Prompt injection - Wikipedia, accessed March 20, 2026, https://en.wikipedia.org/wiki/Prompt_injection
31. LLM01:2025 Prompt Injection - OWASP Gen AI Security Project, accessed March 20, 2026, https://genai.owasp.org/llmrisk/llm01-prompt-injection/
32. Not what you've signed up for: Compromising Real ... - Black Hat, accessed March 20, 2026, https://i.blackhat.com/BH-US-23/Presentations/US-23-Greshake-Not-what-youve-signed-up-for-whitepaper.pdf
33. Indirect prompt injection to LLMs | Fluid Attacks, accessed March 20, 2026, https://fluidattacks.com/blog/indirect-prompt-injection-llms
34. Defending Against Indirect Prompt Injection Attacks With Spotlighting - CEUR-WS.org, accessed March 20, 2026, https://ceur-ws.org/Vol-3920/paper03.pdf
35. Indirect Prompt Injection Threats, accessed March 20, 2026, https://greshake.github.io/
36. Prompt Injections: A Practical Taxonomy of Attack Methods - Pangea Cloud, accessed March 20, 2026, https://pangea.cloud/securebydesign/aiapp-pi-taxonomy/
37. Prompt Injection Attacks | Risks & Protection Strategies - Imperva, accessed March 20, 2026, https://www.imperva.com/learn/application-security/prompt-injection/
38. Open Policy Agent (OPA), accessed March 20, 2026, https://www.openpolicyagent.org/docs/latest/
39. Policy Language - Open Policy Agent, accessed March 20, 2026, https://openpolicyagent.org/docs/policy-language
40. Open Policy Agent (OPA), accessed March 20, 2026, https://openpolicyagent.org/docs
41. NeMo Guardrails | NVIDIA Developer, accessed March 20, 2026, https://developer.nvidia.com/nemo-guardrails
42. Introduction - Guardrails AI, accessed March 20, 2026, https://guardrailsai.com/guardrails/docs
43. NVIDIA NeMo Guardrails Library Developer Guide — NVIDIA NeMo ..., accessed March 20, 2026, https://docs.nvidia.com/nemo/guardrails/latest/index.html
44. Guardrails AI and NVIDIA NeMo Guardrails - A Comprehensive Approach to AI Safety - My Framer Site, accessed March 20, 2026, https://guardrailsai.com/blog/nemoguardrails-integration
45. Release Notes — NVIDIA NeMo Guardrails Library Developer Guide, accessed March 20, 2026, https://docs.nvidia.com/nemo/guardrails/latest/about/release-notes.html
46. Guardrails AI vs NeMo Guardrails | AI Security Comparison - Keywords AI, accessed March 20, 2026, https://www.keywordsai.co/market-map/compare/guardrails-ai-vs-nemo-guardrails

47. Microsoft Presidio: An Open Source Tool Specialized in Personal Information Protection | Mamezou Developer Portal, accessed March 20, 2026, https://developer.mamezou-tech.com/en/blogs/2025/01/04/presidio-intro/
48. Home - Microsoft Presidio, accessed March 20, 2026, https://microsoft.github.io/presidio/anonymizer/
49. Microsoft Presidio Community Version: Open-Source PII Detection and Anonymization Tool, accessed March 20, 2026, https://hoop.dev/blog/microsoft-presidio-community-version-open-source-pii-detection-and-anonymization-tool/
50. Zero Trust Architecture - NIST Technical Series Publications, accessed March 20, 2026, https://nvlpubs.nist.gov/nistpubs/specialpublications/NIST.SP.800-207.pdf
51. What is the NIST SP 800-207 cybersecurity framework? - CyberArk, accessed March 20, 2026, https://www.cyberark.com/what-is/nist-sp-800-207-cybersecurity-framework/
52. Zero Trust Architecture (ZTA) Explained - NIST 800-207, accessed March 20, 2026, https://getnametag.com/newsroom/nist-800-207-zero-trust-architecture-zta-explained
53. The Logical Components of Zero Trust - InterSec Inc., accessed March 20, 2026, https://www.intersecinc.com/blogs/the-logical-components-of-zero-trust
54. NIST SP 800-207 – The Definitive Guide to Zero Trust Architecture - TerraZone, accessed March 20, 2026, https://terrazone.io/nist-sp-800-207/
55. ServiceNow Supports NIST 800-207 Zero-Trust Cybersecurity, accessed March 20, 2026, https://www.servicenow.com/community/secops-articles/servicenow-supports-nist-800-207-zero-trust-cybersecurity/ta-p/3455669
56. Threat Modeling for Drivers - Windows drivers | Microsoft Learn, accessed March 20, 2026, https://learn.microsoft.com/en-us/windows-hardware/drivers/driversecurity/threat-modeling-for-drivers
57. STRIDE model - Wikipedia, accessed March 20, 2026, https://en.wikipedia.org/wiki/STRIDE_model
58. CMS Threat Modeling Handbook | CMS Information Security and Privacy Program, accessed March 20, 2026, https://security.cms.gov/learn/cms-threat-modeling-handbook
59. Threat modeling STRIDE methodology - IriusRisk, accessed March 20, 2026, https://www.iriusrisk.com/resources-blog/threat-modeling-methodology-stride
60. Threat Modeling Process - OWASP Foundation, accessed March 20, 2026, https://owasp.org/www-community/Threat_Modeling_Process
61. Docker Engine security - Docker Docs, accessed March 20, 2026, https://docs.docker.com/engine/security/
62. Docker Security - OWASP Cheat Sheet Series, accessed March 20, 2026, https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html
63. Good guidelines for Securing docker containers and host system? (No remote

access), accessed March 20, 2026, https://www.reddit.com/r/selfhosted/comments/1q9fjfz/good_guidelines_for_securing_docker_containers/
64. FastAPI Best Practices - Auth0, accessed March 20, 2026, https://auth0.com/blog/fastapi-best-practices/
65. Best Practices in FastAPI Architecture: A Complete Guide to Building Scalable, Modern APIs, accessed March 20, 2026, https://zyneto.com/blog/best-practices-in-fastapi-architecture
66. FastAPI Best Practices for Production APIs | by kasim Segun - Medium, accessed March 20, 2026, https://medium.com/@kasimoluwasegun/fastapi-best-practices-for-production-apis-924676d5d134
67. Middleware x Router-Level Dependencies | Auth : r/FastAPI - Reddit, accessed March 20, 2026, https://www.reddit.com/r/FastAPI/comments/1o7qynk/middleware_x_routerlevel_dependencies_auth/
68. FastAPI Auth with Dependency Injection - PropelAuth, accessed March 20, 2026, https://www.propelauth.com/post/fastapi-auth-with-dependency-injection
69. FastAPI dependency vs middleware - python - Stack Overflow, accessed March 20, 2026, https://stackoverflow.com/questions/66632841/fastapi-dependency-vs-middleware
70. JSONL vs NDJSON: Complete Comparison Guide, accessed March 20, 2026, https://jsonltools.com/jsonl-vs-ndjson
71. What is NDJSON? Complete Guide to Newline Delimited JSON - JSONL Tools, accessed March 20, 2026, https://jsonltools.com/what-is-ndjson
72. Use ndjson instead of JSON array - Medium, accessed March 20, 2026, https://medium.com/@mbh023/use-ndjson-instead-of-json-array-c0589972f5fa
73. JSON Parsing for Large Payloads: Balancing Speed, Memory, and Scalability, accessed March 20, 2026, https://towardsdatascience.com/json-parsing-for-large-payloads-balancing-speed-memory-and-scalability/